

Basic (your first LCEL chain) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
model = ChatOpenAI()
parser = StrOutputParser()

chain = prompt | model | parser

result = chain.invoke({"question": "What's 5 + 7?"})
print(result)
```

Expected output:

```
5 + 7 = 12
```

This version works correctly. It's missing an explicit model name and temperature (it relies on `ChatOpenAI()`'s hidden defaults), and it builds the pieces as separate named variables before chaining — fine for a first working version, but not quite what you'd write once the pattern feels natural.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a small `build_chain()` function, with explicit settings

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

def build_chain(model_name: str = "gpt-4o-mini", temperature: float = 0.0):
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    model = ChatOpenAI(model=model_name, temperature=temperature)
    parser = StrOutputParser()
    return prompt | model | parser

if __name__ == "__main__":
    chain = build_chain()
    result = chain.invoke({"question": "What's 5 + 7?"})
    print(f"Answer: {result}")
```

Expected output:

```
Answer: 5 + 7 = 12
```

Difference from Basic: the model name and temperature are explicit parameters instead of hidden defaults — this matters because a model swap or a temperature change should be a visible, one-line decision, not something buried in a library default that might change between versions.

Wrapping the chain-building in a function (`build_chain()`) also means this exact chain can be imported and reused elsewhere, which is exactly what this document's Build Task asks you to do.

Which one should you actually write? Intermediate's single-input chain is all this document's exercises and Build Task need. `chain.batch(inputs, return_exceptions=True)` is worth knowing exists the moment you're processing more than a handful of inputs through the same chain — a CSV of rows, a list of user messages to classify — where losing every result to one bad row

would be a real, avoidable cost, and where requests running concurrently instead of one-by-one genuinely saves time. Not needed for a single-input chain like this one.